

COMPILADORES • PROJETO MINIC

AULA 13

10 exercícios práticos de implementação

Tradução dirigida por sintaxe em Python e C

Atributos • SDDs S-atribuídas e L-atribuídas • dependências • ações semânticas • AST

Ciência da Computação | Projeto MINIC | 2026/2

Orientações gerais

Esta lista reúne dez atividades progressivas de implementação baseadas nos objetivos da Aula 13. Em cada exercício, desenvolva uma versão em Python e outra em C; mantenha as regras semânticas equivalentes e registre diferenças de representação ou gerenciamento de memória quando houver.

Entrega sugerida: código-fonte das duas versões, instruções para executar, testes reproduzíveis e uma breve justificativa das regras de atributos e ações semânticas adotadas.

Qualidade mínima: entradas válidas e inválidas testadas; diagnósticos com posição quando pertinente; nenhuma falha escondida; AST representada por nós, não apenas por concatenação de texto.

Critério de avaliação sugerido: cada exercício vale 10 pontos: correção semântica (4), implementação nas duas linguagens (3), testes (2) e clareza/documentação (1). O docente pode ajustar a pontuação.

Resultados de aprendizagem

- Distinguir atributos sintetizados e herdados e indicar o sentido do fluxo de informação.
- Construir grafos de dependência e justificar uma ordem de avaliação válida.
- Associar regras semânticas a produções e implementar SDDs S-atribuídas e L-atribuídas.
- Construir ASTs incrementalmente durante a análise de expressões e comandos MINIC.
- Relacionar scanner, parser, atributos e ações semânticas, preservando posição e diagnósticos.

Exercício 01 — Calculadora de expressões com atributos sintetizados

Foco da aula: Atributos sintetizados; SDD; avaliação de expressões.

Desafio

Implemente um avaliador para a gramática $E \rightarrow E + T \mid E - T \mid T$; $T \rightarrow T * F \mid T / F \mid F$; $F \rightarrow (E) \mid \text{num}$. Cada produção deve calcular e devolver o valor do não terminal à esquerda.

O que implementar

- Defina as regras semânticas para soma, subtração, multiplicação, divisão, parênteses e números; explicita o atributo val de cada não terminal.
- Implemente a avaliação em Python e em C. Use analisador recursivo descendente ou uma rotina de reduções simuladas, mas mantenha a mesma gramática e a mesma precedência nas duas versões.
- Retorne um resultado numérico e rejeite divisão por zero com diagnóstico claro, sem confundir esse erro de execução semântica com erro de sintaxe.
- Inclua uma opção de rastreamento que mostre, em ordem, as regras aplicadas e os valores sintetizados.

Requisitos por linguagem

Python: Crie funções por nível da gramática (`parse_E`, `parse_T`, `parse_F`) que retornem o valor calculado e avancem sobre uma sequência de tokens.

C: Implemente as mesmas funções com um vetor de tokens e índice de leitura; passe o índice por referência ou mantenha-o em uma estrutura Parser.

Testes mínimos

$3 + 5 \rightarrow 8$; $2 + 3 * 4 \rightarrow 14$; $(2 + 3) * 4 \rightarrow 20$; $18 / 3 / 2 \rightarrow 3$ (associatividade à esquerda); $1 / 0 \rightarrow$ erro semântico.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 02 — Grafo de dependências e avaliador topológico

Foco da aula: Instâncias de atributos; dependências; ordenação topológica; ciclos.

Desafio

Construa um pequeno motor que recebe atributos e dependências entre eles, como as necessárias para avaliar $3 * 5$, e executa as ações em uma ordem válida.

O que implementar

- Modele cada atributo como um identificador único (por exemplo, `num1.lexval`, `F1.val`, `T.val`) e represente cada dependência como uma aresta dirigida.
- Implemente ordenação topológica (Kahn ou DFS com marcação de estados) e execute ações quando todos os pré-requisitos estiverem disponíveis.
- Detecte ciclos e informe quais atributos ficaram bloqueados; não produza um valor final parcial como se fosse válido.
- Mostre a sequência de avaliação e o valor final para a expressão $3 * 5$.

Requisitos por linguagem

Python: Use dicionários para valores e listas/conjuntos para adjacências; implemente uma função `evaluate(graph, actions)` que retorne ordem e resultado.

C: Use vetores de nós/arestas e uma fila para o algoritmo de Kahn; documente limites máximos ou aloque estruturas dinamicamente.

Testes mínimos

Grafo acíclico de $3 * 5$ deve avaliar para 15; uma cadeia de dependências deve respeitar a ordem; um ciclo $A \rightarrow B \rightarrow C \rightarrow A$ deve ser detectado.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 03 — Construtor de AST com SDD S-atribuída

Foco da aula: Atributos sintetizados; reduções; nós-folha e nós internos.

Desafio

Implemente uma tradução em que cada produção devolve um ponteiro/referência para uma expressão AST. As ações só podem usar atributos sintetizados dos filhos.

O que implementar

- Defina os nós Number, Identifier, Unary e Binary, incluindo operador e filhos; forneça construtores e uma função de liberação/limpeza em C.
- Escreva as regras semânticas para id, num, (E), -E e E op E, registrando para cada produção o nó que ela sintetiza.
- Construa a árvore de $a - 4 + c$ respeitando a associatividade à esquerda: a operação de subtração deve ser filha esquerda da soma.
- Implemente uma impressão prefixa totalmente parentizada e uma travessia de avaliação para expressões numéricas.

Requisitos por linguagem

Python: Use classes ou dataclasses imutáveis para os nós; faça cada rotina retornar um objeto AST e evite montar a árvore como texto concatenado.

C: Use enum para o tipo de nó e uma union/struct; defina quem é dono de cada filho e trate falhas de alocação sem vazamentos.

Testes mínimos

$a - 4 + c$ deve imprimir $(+ (- a 4) c)$; $2 * (3 + 4)$ deve preservar o agrupamento e avaliar para 14 quando não houver identificadores.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 04 — Propagação de tipo herdado em listas de identificadores

Foco da aula: Atributos herdados; SDD L-atribuída; contexto à esquerda.

Desafio

Implemente declarações da forma tipo lista ;, por exemplo `int a, b, c;`. O atributo `type` produzido pelo tipo deve chegar a cada identificador da lista.

O que implementar

- Especifique as produções $D \rightarrow T L$; $T \rightarrow \text{int} \mid \text{float} \mid \text{char}$; $L \rightarrow \text{id} \mid L, \text{id}$ e as regras para `L.inh` ou para inserção dos identificadores na tabela de símbolos.
- Garanta que a propagação siga da esquerda para a direita e explique por que as dependências são compatíveis com uma SDD L-atribuída.
- Armazene nome, tipo e posição de origem de cada identificador; rejeite uma redeclaração no mesmo escopo.
- Produza uma listagem final da tabela de símbolos em ordem determinística.

Requisitos por linguagem

Python: Passe o tipo como argumento explícito para a rotina que analisa a lista e acumule os registros numa tabela de símbolos.

C: Passe o tipo como parâmetro da função de análise e implemente a tabela como vetor dinâmico ou lista ligada com busca de duplicatas.

Testes mínimos

`int a, b;` deve registrar `a:int` e `b:int`; `float x;` deve registrar `x:float`; `int a, a;` deve apontar a segunda ocorrência como redeclaração.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 05 — Parser descendente com contexto herdado

Foco da aula: L-atribuídas; parâmetros de procedimentos; ações durante análise descendente.

Desafio

Amplie o exercício anterior para analisar blocos aninhados. O parser deve transportar o escopo atual como contexto herdado e permitir sombreamento controlado.

O que implementar

- Defina uma estrutura de escopo com referência ao escopo pai e uma regra clara para entrada e saída de blocos.
- Implemente declarações locais e busca de identificadores: primeiro no escopo atual, depois nos ancestrais.
- Faça o contexto chegar às rotinas descendentes por parâmetro ou estrutura explícita; não use variáveis globais ocultas para carregar o escopo.
- Registre na AST a declaração resolvida ou o nome do escopo, sem remover a informação de linha e coluna.

Requisitos por linguagem

Python: Use objetos Scope com parent, symbols e métodos declare/lookup; passe o escopo ao analisar cada bloco.

C: Use struct Scope com parent e tabela de símbolos; implemente enter_scope/leave_scope com política de propriedade documentada.

Testes mínimos

Uma variável declarada no bloco externo deve ser encontrada no interno; uma declaração interna pode sombreá-la; um nome ausente deve gerar diagnóstico na posição de uso.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 06 — AST de expressões MINIC respeitando precedência

Foco da aula: Ações semânticas; construção incremental de AST; forma da gramática.

Desafio

Integre um scanner mínimo já existente ou fornecido pelo docente e implemente um parser de expressões MINIC que construa AST diretamente durante o reconhecimento.

O que implementar

- Aceite identificadores, inteiros, parênteses, unários – e !, operadores * / %, + –, relacionais, igualdade, && e ||.
- Defina níveis de precedência e associatividade; associe cada produção a uma ação que cria Identifier, Integer, Unary ou Binary.
- Implemente uma serialização determinística da AST e preserve lexema, linha e coluna nas folhas.
- Compare a estrutura resultante com as árvores esperadas para expressões com operadores de níveis diferentes.

Requisitos por linguagem

Python: Faça o parser consumir objetos Token do scanner; cada método de precedência retorna um nó AST. A camada léxica não deve ser duplicada no parser.

C: Consuma a interface de tokens do scanner por uma função next_token/lookahead; devolva nós AST alocados dinamicamente.

Testes mínimos

$a + b * c \rightarrow (+ a (* b c))$; $(a + b) * c \rightarrow (* (+ a b) c)$; $a - b - c$ deve associar à esquerda; $!a == b$ deve obedecer à precedência especificada.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 07 — Ações semânticas para comandos e fluxo de controle

Foco da aula: Ações semânticas em produções; AST de comandos MINIC.

Desafio

Implemente a construção de nós AST para comandos de um subconjunto MINIC, sem gerar ainda código executável.

O que implementar

- Reconheça bloco, atribuição, if/else, while, return e expressão seguida de ponto e vírgula.
- Defina os campos e invariantes dos nós Block, Assign, If, While e Return; em if sem else, represente explicitamente a ausência do ramo alternativo.
- Anexe ações às produções para que filhos e listas de comandos sejam associados na ordem correta.
- Imprima a AST e valide que blocos vazios e comandos aninhados não perdem a estrutura.

Requisitos por linguagem

Python: Use uma hierarquia de classes de AST ou dataclasses; represente listas de comandos como listas, não como strings concatenadas.

C: Defina structs discriminadas por enum; forneça construtores para cada comando e uma rotina recursiva de destruição da árvore.

Testes mínimos

`while (x) { x = x - 1; }` deve conter While com Block; `if (a) b=1; else b=2;` deve manter os dois ramos; `return;` deve representar expressão opcional.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 08 — Validação de tipos como atributos sintetizados

Foco da aula: Atributos semânticos; composição de tipos; separação entre sintaxe e semântica.

Desafio

Após construir a AST de expressões, calcule um atributo `type` para cada nó e reporte incompatibilidades sem alterar a árvore sintática.

O que implementar

- Defina os tipos `int`, `float` e `bool` e regras para literais, identificadores, operadores aritméticos, relacionais e lógicos.
- Faça `Binary.type` depender dos atributos de tipo dos filhos; estabeleça conversões permitidas e regras de resultado.
- Acumule diagnósticos semânticos com posição e mensagem; preserve a AST mesmo quando houver tipos incompatíveis.
- Separe claramente falha sintática de falha semântica na interface e nos códigos de retorno.

Requisitos por linguagem

Python: Implemente uma passagem recursiva `check_expr(node, env)` que devolve `Type` e acrescenta erros estruturados a uma lista.

C: Implemente uma função recursiva com retorno `enum Type` e um vetor de `Diagnostic`; evite encerrar o processo dentro da função de verificação.

Testes mínimos

`3 + 4` deve ser `int`; `3.0 + 4` deve ser `float` sob a política de promoção definida; `true && false` deve ser `bool`; `3 + true` deve gerar erro semântico localizado.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 09 — Execução segura de ações e tratamento de erros

Foco da aula: Pré-condições das ações; dependências; integridade da AST e diagnósticos.

Desafio

Adicione tratamento robusto a um parser que executa ações semânticas. Um erro em uma produção não deve deixar nós parcialmente ligados nem publicar uma AST incompleta como válida.

O que implementar

- Defina objetos Diagnostic com categoria, mensagem, lexema, linha e coluna para erros léxicos, sintáticos e semânticos.
- Faça cada ação verificar suas pré-condições: atributos necessários disponíveis, filhos válidos e operador reconhecido.
- Ao falhar a construção de um nó, libere recursos temporários e marque o resultado como inválido; não confunda nó ausente com uma AST válida vazia.
- Implemente recuperação em pontos de sincronização (por exemplo, ponto e vírgula e chave de fechamento) e acumule mais de um diagnóstico quando for seguro.

Requisitos por linguagem

Python: Use exceções específicas apenas para interromper uma produção quando necessário; mantenha os diagnósticos numa lista e não capture exceções genéricas silenciosamente.

C: Propague códigos de erro ou ponteiros nulos com estado explícito; em cada caminho de falha, destrua os nós temporários e retorne status diferente de sucesso.

Testes mínimos

Teste caractere inválido, token inesperado, ponto e vírgula ausente, duas declarações defeituosas separadas por sincronização e ausência de AST marcada como válida após erro.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Exercício 10 — Mini pipeline scanner–parser–atributos–AST

Foco da aula: Integração; SDD; AST; atributos; testes diferenciais.

Desafio

Como projeto integrador, implemente um subconjunto MINIC que percorra o fluxo completo da aula: scanner → tokens → parser → ações/atributos → AST.

O que implementar

- Suporte declarações simples, atribuições, expressões aritméticas, if, while e return; declare explicitamente o subconjunto da gramática implementado.
- Use o scanner como módulo independente e faça o parser consumir tokens com tipo, lexema e posição. O scanner não deve ser copiado para dentro do parser.
- Construa AST com nós tipados e atributos sintetizados; use ao menos um atributo herdado para propagar o tipo ou escopo durante uma declaração.
- Execute a mesma bateria de testes nas versões Python e C; compare status, diagnósticos normalizados e serialização da AST.
- Documente as regras semânticas, dependências relevantes, limitações e comandos de compilação/execução.

Requisitos por linguagem

Python: Separe módulos scanner.py, ast_nodes.py e parser.py; forneça uma função parse(source) e uma CLI que leia arquivo ou stdin.

C: Separe scanner.h/.c, ast.h/.c e parser.h/.c; forneça uma função parse(TokenStream*, ...) e uma CLI que reporte erros em stderr.

Testes mínimos

Inclua pelo menos cinco programas válidos e cinco inválidos; cubra precedência, associação, posição de erro, declaração e comando aninhado; ambas as linguagens devem concordar nos resultados.

Evidência a entregar: inclua uma saída de execução ou teste automatizado que demonstre o comportamento esperado e explique pelo menos uma decisão de atributos/ações semânticas.

Referências e alinhamento

Conteúdo elaborado a partir da apostila “Aula 13 — Tradução dirigida por sintaxe” e dos slides da Aula 13 fornecidos para o Projeto MINIC. Os tópicos cobertos incluem atributos sintetizados e herdados, definições dirigidas por sintaxe, grafos de dependência e avaliação topológica, SDDs S-atribuídas e L-atribuídas, ações semânticas e construção de AST.

Referência bibliográfica: AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. Compiladores: princípios, técnicas e ferramentas. 2. ed. Capítulo 5, especialmente seções 5.1 e 5.2.

Os exercícios são propostas originais de implementação e não reproduzem trechos extensos do livro.